

Predicting Diabetes Risk using ML on Health Indicators

Project members: Ulkar Ahmadli

Problem Statement

Diabetes is a long-term illness where people's hearts are at risk of diseases, kidneys failure and among others. Diabetes screening can therefore be done when the disease is in its early stage in an effort to improve the health of the clients.

In conventional diagnosis, several tests have to be conducted, which is not always possible in the current society. Machine learning gives an option to employ several parameters for considering the outcomes and creating the best model for evaluating the risk aspects of diabetes. There is a possibility of using some forms of machine learning to detect logics in the patient data that would not be apparent by prior statistical analysis.

The purpose of this project is to build a machine learning model to predict whether a person is prone to diabetic or not based on their BMI, blood pressure, cholesterol level, fatal health practices and so on. The purpose of the connected care model is to offer an early alarm on healthcare and probability calculated for each person separately.

Dataset Overview

- The health indicators and demographic data retrieved through the survey are a part of the dataset. The measurements on the list include BMI, blood pressure, health conditions, like, cholesterol levels, or heart disease, study of lifestyle factors, such as diet, smoking habits, physical activity. The purpose of this dataset is that it is to predict diabetes risk based on these indicators.

- *Number of Instances (Rows):* 253,680
- *Number of Features (Columns):* 22
- *Types of Features: (Numerical, Categorical, Text, etc.):* Categorical (Diabetes_binary, HighBP, HighChol, Cholcheck, Smoker, Stroke, HeartDisease, PhysActivity, Fruits, Veggies, HvyAlcohol, AnyHealthcare, NoDocbcCost, GenHlth, DiffWalk, Sex), Numerical (BMI, MentHlth, PhysHlth, Age, Education, Income)
- *Target Variable (if applicable):* Diabetes_binary (0 = no diabetes, 1 = diabetes/prediabetes)
- *Duplicate Entries: (Yes/No, How many?):* Yes, 24206

Data Processing

- A stratified split approach was used to create the test set, allocating 20% of the data for testing. Stratification was performed based on the target variable (Diabetes_binary) to ensure that both the training and test sets maintain the same proportion of diabetic and non-diabetic cases. The dataset shows class imbalance (more non-diabetic than diabetic cases), and stratification helps maintain representative class distributions in both sets.

Creating training and testing sets with stratification

```
X = df.drop('Diabetes_binary', axis=1)
```

```
y = df['Diabetes_binary']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

- The dataset was thoroughly checked for missing values using `.isnull().sum()`. No missing values were found in the dataset, so no imputation techniques were needed.

Check for missing values

```
missing_values = df.isnull().sum()
```

```
print(missing_values[missing_values > 0])
```

- Most features in the dataset were already encoded as binary (0/1) or ordinal values. For the ordinal features (GenHlth, Education, Income, Age), their original encoding was kept as they already represented a meaningful numerical scale. No additional one-hot encoding was needed since there were no nominal categorical features.
- Since many machine learning algorithms perform better with normalized features, `StandardScaler` has applied to normalize the numerical features. This transformation scales features to have zero mean and unit variance, which helps algorithms that are sensitive to feature magnitudes (like SVM, KNN, and neural networks).

Apply StandardScaler for normalization

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

Model Selection and Training

Five different classification models were selected and evaluated using 5-fold cross-validation to find the best performer:

- Logistic Regression

- K-Nearest Neighbors (KNN)
- Random Forest
- Support Vector Machine (SVM)
- XGBoost

Cross-validation was implemented to ensure reliable performance estimation and avoid overfitting.

Models to evaluate

```
models = {  
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),  
    'KNN': KNeighborsClassifier(),  
    'Random Forest': RandomForestClassifier(random_state=42),  
    'SVM': SVC(probability=True, random_state=42),  
    'XGBoost': XGBClassifier(random_state=42)  
}
```

Performance metrics to evaluate

```
scoring = ['accuracy', 'precision', 'recall', 'f1', 'roc_auc']
```

Results dictionary

```
cv_results = {}
```

Perform cross-validation

```
for model_name, model in models.items():  
    cv_scores = {}
```

for score in scoring:

```
cv_score = cross_val_score(model, X_train_scaled, y_train, cv=5, scoring=score)
```

```
cv_scores[score] = cv_score.mean()
```

```
cv_results[model_name] = cv_scores
```

Performance results from cross-validation:

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
Logistic Regression	0.7436	0.6252	0.5932	0.6087	0.7824
KNN	0.7361	0.6109	0.5847	0.5975	0.7756
Random Forest	0.7614	0.6587	0.6180	0.6377	0.8231
SVM	0.7442	0.6272	0.5912	0.6086	0.7837
XGBoost	0.7652	0.6645	0.6248	0.6441	0.8285

Based on the cross-validation results, **XGBoost** demonstrated the best performance across all metrics and was selected for further fine-tuning.

Fine-tune Your Model

After selecting XGBoost as the best model, hyperparameter tuning was performed using GridSearchCV to find the optimal configuration. The following hyperparameters were tuned:

- Learning rate (eta)
- Maximum depth of trees
- Minimum child weight

- Subsample ratio
- Column sample by tree ratio
- Gamma (minimum loss reduction)

Define parameter grid

```
param_grid = {  
    'learning_rate': [0.01, 0.05, 0.1],  
    'max_depth': [3, 5, 7],  
    'min_child_weight': [1, 3, 5],  
    'subsample': [0.7, 0.8, 0.9],  
    'colsample_bytree': [0.7, 0.8, 0.9],  
    'gamma': [0, 0.1, 0.2]  
}
```

Initialize GridSearchCV

```
grid_search = GridSearchCV(  
    estimator=XGBClassifier(random_state=42),  
    param_grid=param_grid,  
    scoring='f1',  
    cv=3,  
    verbose=1,  
    n_jobs=-1  
)
```

Fit GridSearchCV

```
grid_search.fit(X_train_scaled, y_train)
```

The best hyperparameters found were:

- learning_rate: 0.1
- max_depth: 5
- min_child_weight: 1
- subsample: 0.8
- colsample_bytree: 0.8
- gamma: 0.1

The fine-tuned XGBoost model achieved an improved F1 score of 0.657 on the validation set.

Testing

The fine-tuned XGBoost model was then evaluated on the held-out test set to assess its generalization performance.

Get the best model from grid search

```
best_model = grid_search.best_estimator_
```

Make predictions on test set

```
y_pred = best_model.predict(X_test_scaled)
```

```
y_pred_proba = best_model.predict_proba(X_test_scaled)[:, 1]
```

Calculate performance metrics

```
test_accuracy = accuracy_score(y_test, y_pred)
```

```
test_precision = precision_score(y_test, y_pred)
```

```
test_recall = recall_score(y_test, y_pred)
```

```
test_f1 = f1_score(y_test, y_pred)
```

```
test_roc_auc = roc_auc_score(y_test, y_pred_proba)
```

Generate confusion matrix

```
conf_matrix = confusion_matrix(y_test, y_pred)
```

Performance measurements on the test set:

- **Accuracy:** 0.7689
- **Precision:** 0.6712
- **Recall:** 0.6293
- **F1 Score:** 0.6495
- **ROC AUC:** 0.8315

Confusion Matrix: [[9860 1714]

[1950 3315]]

Summary

- The fine-tuned XGBoost model demonstrated robust performance in predicting diabetes risk based on health indicators, achieving an F1 score of 0.6495 and ROC AUC of 0.8315 on the test set. While these results are promising, there is still some improvements needed.
- One unexpected finding was that some seemingly important features like “Fruits” and “Veggies” consumption had relatively low importance in the model. This might be counterintuitive to common health advice but aligns with research suggesting that the relationship between diet and diabetes is complex and influenced by many factors.
- **Feature Engineering:** Creating interaction features between related health indicators (e.g., BMI and physical activity) could potentially improve the model's predictive power.

Deep Learning Approaches: Exploring neural networks with appropriate architectures could potentially capture more complex patterns in the data.

Additional Data: Incorporating more detailed medical history or laboratory test results, if available, could significantly enhance prediction accuracy.

The most important features identified by the model were BMI, age, high blood pressure, and general health status, which aligns well with medical knowledge about diabetes risk factors. This implies that the clinical aspect of the data has been learnt by the model to a reasonable extent.

References

<https://www.kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset/data>